

Watch 467 Like Fork Star

The agent harness performance optimization system. Skills, instincts, memory, security, and research-first development for Claude Code, Codex, Opencode, Cursor and beyond.

ecc.tools

MIT license

Code of conduct

Contributing

90.3k stars 11.8k forks 467 watching 2 Branches 12 Tags Activity

Public repository

2 Branches 12 Tags Go to file Go to file Add file Code

affaan-m	fix: skip flaky observer shell test on Windows (#722) ✓	7ccfda9 · 6 hours ago
.agents/skills	feat(rules): add C# language support (#704)	8 hours ago
.claude-plugin	docs: clarify custom endpoint support	last week
.claude	feat(rules): add C# language support (#704)	8 hours ago
.codex	feat(rules): add C# language support (#704)	8 hours ago
.cursor	feat: add block-no-verify hook for Claude Co...	11 hours ago
.github	chore(config): governance and config found...	4 days ago
.opencode	chore: prepare v1.9.0 release (#666)	13 hours ago
agents	feat(agents): add flutter-reviewer agent and ...	6 hours ago
assets/images	Revert "feat(ecc): prune plugin 43 → 12 items...	last month
commands	feat(skills): add rules-distill skill (rebased #5...	11 hours ago
contexts	Revert "feat(ecc): prune plugin 43 → 12 items...	last month
docs	fix: harden windows CI tests and markdown ...	10 hours ago
examples	feat: add laravel skills (#420)	4 days ago
hooks	feat: add MCP health-check hook (#711)	7 hours ago
manifests	feat: implement --with/--without selective ins...	12 hours ago
mcp-configs	Add Claude DevFleet multi-agent orchestrati...	4 days ago
plugins	chore: replace external repo links with @use...	2 weeks ago
rules	feat(rules): add C# language support (#704)	8 hours ago
schemas	feat: implement --with/--without selective ins...	12 hours ago
scripts	fix: codex sync merges AGENTS.md instead ...	6 hours ago
skills	feat(agents): add flutter-reviewer agent and ...	6 hours ago
tests	fix: skip flaky observer shell test on Window...	6 hours ago

 .env.example	chore(config): governance and config found...	4 days ago
 .gitignore	chore(config): governance and config found...	4 days ago
 .markdownlint.json	chore(config): governance and config found...	4 days ago
 .npmignore	chore: rename opencode plugin to ecc-unive...	last month
 .prettierrc	chore: add .prettierrc for consistent code for...	2 weeks ago
 .tool-versions	chore(config): governance and config found...	4 days ago
 AGENTS.md	fix: update catalog counts for flutter-reviewe...	6 hours ago
 CHANGELOG.md	chore: prepare v1.9.0 release (#666)	13 hours ago
 CLAUDE.md	docs: add CLAUDE.md for Claude Code guid...	last month
 CODE_OF_CONDUCT.md	Fix markdownlint errors from merge of affaa...	last week
 CONTRIBUTING.md	feat(commands): add /docs; feat(mcp-confi...	4 days ago
 LICENSE	docs: add missing MIT LICENSE file	2 months ago
 README.md	fix: update catalog counts for flutter-reviewe...	6 hours ago
 README.zh-CN.md	Merge pull request #392 from hahmee/docs...	last week
 SPONSORING.md	docs: add sponsorship playbook and monthl...	2 weeks ago
 SPONSORS.md	docs: strengthen sponsor optics with live m...	2 weeks ago
 TROUBLESHOOTING.md	docs: tighten troubleshooting safety guidance	last week
 VERSION	chore(config): governance and config found...	4 days ago
 commitlint.config.js	feat: add comprehensive CI/CD pipeline	2 months ago
 eslint.config.js	fix: add global ignores to ESLint config for di...	last month
 install.ps1	Add PowerShell installer wrapper and updat...	4 days ago
 install.sh	feat: orchestration harness, selective install, ...	last week
 package-lock.json	chore(deps-dev): bump flatted (#675)	11 hours ago
 package.json	chore: prepare v1.9.0 release (#666)	13 hours ago
 the-longform-guide.md	Fix markdownlint errors from merge of affaa...	last week
 the-openclaw-guide.md	feat: add security guides and sanitize extern...	3 weeks ago
 the-security-guide.md	Revert "fix: remove internal sponsor/partner ...	2 weeks ago
 the-shortform-guide.md	Fix markdownlint errors from merge of affaa...	last week

README Code of conduct Contributing MIT license



Language: English | [简体中文](#) | [繁體中文](#) | [日本語](#) | [한국어](#)

🔗 Everything Claude Code

stars 90k forks 12k contributors 101  ecc-universal weekly downloads 1.8k/week  ecc-agentshield weekly downloads 667/week  GitHub App 150 installs
license MIT  Shell  TypeScript  Python  Go  Java  Perl  Markdown

🌐 [Language](#) / [语言](#) / [語言](#)

[English](#) | [简体中文](#) | [繁體中文](#) | [日本語](#) | [한국어](#)

The performance optimization system for AI agent harnesses. From an Anthropic hackathon winner.

Not just configs. A complete system: skills, instincts, memory optimization, continuous learning, security scanning, and research-first development. Production-ready agents, hooks, commands, rules, and MCP configurations evolved over 10+ months of intensive daily use building real products.

Works across **Claude Code**, **Codex**, **Cowork**, and other AI agent harnesses.

[🔗](#) **The Guides**

This repo is the raw code only. The guides explain everything.



Shorthand Guide	Longform Guide
Setup, foundations, philosophy. Read this first.	Token optimization, memory persistence, evals, parallelization.

Topic	What You'll Learn
Token Optimization	Model selection, system prompt slimming, background processes
Memory Persistence	Hooks that save/load context across sessions automatically
Continuous Learning	Auto-extract patterns from sessions into reusable skills
Verification Loops	Checkpoint vs continuous evals, grader types, pass@k metrics
Parallelization	Git worktrees, cascade method, when to scale instances
Subagent Orchestration	The context problem, iterative retrieval pattern

[🔗](#) **What's New**

[🔗](#) **v1.9.0 — Selective Install & Language Expansion (Mar 2026)**

- **Selective install architecture** — Manifest-driven install pipeline with `install-plan.js` and `install-apply.js` for targeted component installation. State store tracks what's installed and enables incremental updates.

- **6 new agents** — `typescript-reviewer`, `pytorch-build-resolver`, `java-build-resolver`, `java-reviewer`, `kotlin-reviewer`, `kotlin-build-resolver` expand language coverage to 10 languages.
- **New skills** — `pytorch-patterns` for deep learning workflows, `documentation-lookup` for API reference research, `bun-runtime` and `nextjs-turbopack` for modern JS toolchains, plus 8 operational domain skills and `mcp-server-patterns`.
- **Session & state infrastructure** — SQLite state store with query CLI, session adapters for structured recording, skill evolution foundation for self-improving skills.
- **Orchestration overhaul** — Harness audit scoring made deterministic, orchestration status and launcher compatibility hardened, observer loop prevention with 5-layer guard.
- **Observer reliability** — Memory explosion fix with throttling and tail sampling, sandbox access fix, lazy-start logic, and re-entrancy guard.
- **12 language ecosystems** — New rules for Java, PHP, Perl, Kotlin/Android/KMP, C++, and Rust join existing TypeScript, Python, Go, and common rules.
- **Community contributions** — Korean and Chinese translations, InsAlts security hook, biome hook optimization, VideoDB skills, Evos operational skills, PowerShell installer, Antigravity IDE support.
- **CI hardening** — 19 test failure fixes, catalog count enforcement, install manifest validation, and full test suite green.

🔗 v1.8.0 — Harness Performance System (Mar 2026)

- **Harness-first release** — ECC is now explicitly framed as an agent harness performance system, not just a config pack.
- **Hook reliability overhaul** — SessionStart root fallback, Stop-phase session summaries, and script-based hooks replacing fragile inline one liners.
- **Hook runtime controls** — `ECC_HOOK_PROFILE=minimal|standard|strict` and `ECC_DISABLED_HOOKS=...` for runtime gating without editing hook files.
- **New harness commands** — `/harness-audit`, `/loop-start`, `/loop-status`, `/quality-gate`, `/model-route`.
- **NanoClaw v2** — model routing, skill hot-load, session branch/search/export/compact/metrics.
- **Cross-harness parity** — behavior tightened across Claude Code, Cursor, OpenCode, and Codex app/CLI.
- **997 internal tests passing** — full suite green after hook/runtime refactor and compatibility updates.

🔗 v1.7.0 — Cross-Platform Expansion & Presentation Builder (Feb 2026)

- **Codex app + CLI support** — Direct `AGENTS.md`-based Codex support, installer targeting, and Codex docs
- `frontend-slides` skill — Zero-dependency HTML presentation builder with PPTX conversion guidance and strict viewport-fit rules
- **5 new generic business/content skills** — `article-writing`, `content-engine`, `market-research`, `investor-materials`, `investor-outreach`
- **Broader tool coverage** — Cursor, Codex, and OpenCode support tightened so the same repo ships cleanly across all major harnesses
- **992 internal tests** — Expanded validation and regression coverage across plugin, hooks, skills, and packaging

🔗 v1.6.0 — Codex CLI, AgentShield & Marketplace (Feb 2026)

- **Codex CLI support** — New `/codex-setup` command generates `codex.md` for OpenAI Codex CLI compatibility
- **7 new skills** — `search-first`, `swift-actor-persistence`, `swift-protocol-di-testing`, `regex-vs-llm-structured-text`, `content-hash-cache-pattern`, `cost-aware-llm-pipeline`, `skill-stocktake`
- **AgentShield integration** — `/security-scan` skill runs AgentShield directly from Claude Code; 1282 tests, 102 rules
- **GitHub Marketplace** — ECC Tools GitHub App live at github.com/marketplace/ecc-tools with free/pro/enterprise tiers
- **30+ community PRs merged** — Contributions from 30 contributors across 6 languages
- **978 internal tests** — Expanded validation suite across agents, skills, commands, hooks, and rules

🔗 v1.4.1 — Bug Fix (Feb 2026)

- **Fixed instinct import content loss** — `parse_instinct_file()` was silently dropping all content after frontmatter (Action, Evidence, Examples sections) during `/instinct-import`. Fixed by community contributor @ericcai0814 ([#148](#), [#161](#))

🔗 v1.4.0 — Multi-Language Rules, Installation Wizard & PM2 (Feb 2026)

- **Interactive installation wizard** — New `configure-ecc` skill provides guided setup with merge/overwrite detection
- **PM2 & multi-agent orchestration** — 6 new commands (`/pm2`, `/multi-plan`, `/multi-execute`, `/multi-backend`, `/multi-frontend`, `/multi-workflow`) for managing complex multi-service workflows

- **Multi-language rules architecture** — Rules restructured from flat files into `common/` + `typescript/` + `python/` + `golang/` directories. Install only the languages you need
- **Chinese (zh-CN) translations** — Complete translation of all agents, commands, skills, and rules (80+ files)
- **GitHub Sponsors support** — Sponsor the project via GitHub Sponsors
- **Enhanced CONTRIBUTING.md** — Detailed PR templates for each contribution type

🔗 v1.3.0 — OpenCode Plugin Support (Feb 2026)

- **Full OpenCode integration** — 12 agents, 24 commands, 16 skills with hook support via OpenCode's plugin system (20+ event types)
- **3 native custom tools** — `run-tests`, `check-coverage`, `security-audit`
- **LLM documentation** — `llms.txt` for comprehensive OpenCode docs

🔗 v1.2.0 — Unified Commands & Skills (Feb 2026)

- **Python/Django support** — Django patterns, security, TDD, and verification skills
- **Java Spring Boot skills** — Patterns, security, TDD, and verification for Spring Boot
- **Session management** — `/sessions` command for session history
- **Continuous learning v2** — Instinct-based learning with confidence scoring, import/export, evolution

See the full changelog in [Releases](#).

🔗 🚀 Quick Start

Get up and running in under 2 minutes:

🔗 Step 1: Install the Plugin

```
# Add marketplace
/plugin marketplace add affaan-m/everything-claude-code

# Install plugin
/plugin install everything-claude-code@everything-claude-code
```



🔗 Step 2: Install Rules (Required)

⚠️ **Important:** Claude Code plugins cannot distribute `rules` automatically. Install them manually:

```
# Clone the repo first
git clone https://github.com/affaan-m/everything-claude-code.git
cd everything-claude-code

# Install dependencies (pick your package manager)
npm install          # or: pnpm install | yarn install | bun install

# macOS/Linux
./install.sh typescript # or python or golang or swift or php
# ./install.sh typescript python golang swift php
# ./install.sh --target cursor typescript
# ./install.sh --target antigravity typescript

# Windows PowerShell
.\install.ps1 typescript # or python or golang or swift or php
# .\install.ps1 typescript python golang swift php
# .\install.ps1 --target cursor typescript
# .\install.ps1 --target antigravity typescript

# npm-installed compatibility entrypoint also works cross-platform
npx ecc-install typescript
```



For manual install instructions see the README in the `rules/` folder.

🔗 Step 3: Start Using

```
# Try a command (plugin install uses namespaced form)
/everything-claude-code:plan "Add user authentication"

# Manual install (Option 2) uses the shorter form:
# /plan "Add user authentication"

# Check available commands
/plugin list everything-claude-code@everything-claude-code
```



🌟 **That's it!** You now have access to 28 agents, 116 skills, and 59 commands.

🌐 Cross-Platform Support

This plugin now fully supports **Windows, macOS, and Linux**, alongside tight integration across major IDEs (Cursor, OpenCode, Antigravity) and CLI harnesses. All hooks and scripts have been rewritten in Node.js for maximum compatibility.

🔗 Package Manager Detection

The plugin automatically detects your preferred package manager (npm, pnpm, yarn, or bun) with the following priority:

1. **Environment variable:** `CLAUDE_PACKAGE_MANAGER`
2. **Project config:** `.claude/package-manager.json`
3. **package.json:** `packageManager` field
4. **Lock file:** Detection from `package-lock.json`, `yarn.lock`, `pnpm-lock.yaml`, or `bun.lockb`
5. **Global config:** `~/.claude/package-manager.json`
6. **Fallback:** First available package manager

To set your preferred package manager:

```
# Via environment variable
export CLAUDE_PACKAGE_MANAGER=pnpm

# Via global config
node scripts/setup-package-manager.js --global pnpm

# Via project config
node scripts/setup-package-manager.js --project bun

# Detect current setting
node scripts/setup-package-manager.js --detect
```



Or use the `/setup-pm` command in Claude Code.

🔗 Hook Runtime Controls

Use runtime flags to tune strictness or disable specific hooks temporarily:

```
# Hook strictness profile (default: standard)
export ECC_HOOK_PROFILE=standard

# Comma-separated hook IDs to disable
export ECC_DISABLED_HOOKS="pre:bash:tmux-reminder,post:edit:typecheck"
```



🔗 📦 What's Inside

This repo is a **Claude Code plugin** - install it directly or copy components manually.



```
everything-claude-code/
|-- .claude-plugin/      # Plugin and marketplace manifests
|   |-- plugin.json      # Plugin metadata and component paths
|   |-- marketplace.json # Marketplace catalog for /plugin marketplace add
|
|-- agents/              # 28 specialized subagents for delegation
|   |-- planner.md       # Feature implementation planning
|   |-- architect.md     # System design decisions
|   |-- tdd-guide.md     # Test-driven development
|   |-- code-reviewer.md # Quality and security review
|   |-- security-reviewer.md # Vulnerability analysis
|   |-- build-error-resolver.md
|   |-- e2e-runner.md    # Playwright E2E testing
|   |-- refactor-cleaner.md # Dead code cleanup
|   |-- doc-updater.md   # Documentation sync
|   |-- docs-lookup.md   # Documentation/API lookup
|   |-- chief-of-staff.md # Communication triage and drafts
|   |-- loop-operator.md # Autonomous loop execution
|   |-- harness-optimizer.md # Harness config tuning
|   |-- cpp-reviewer.md  # C++ code review
|   |-- cpp-build-resolver.md # C++ build error resolution
|   |-- go-reviewer.md   # Go code review
|   |-- go-build-resolver.md # Go build error resolution
|   |-- python-reviewer.md # Python code review
|   |-- database-reviewer.md # Database/Supabase review
|   |-- typescript-reviewer.md # TypeScript/JavaScript code review
|   |-- java-reviewer.md # Java/Spring Boot code review
|   |-- java-build-resolver.md # Java/Maven/Gradle build errors
|   |-- kotlin-reviewer.md # Kotlin/Android/KMP code review
|   |-- kotlin-build-resolver.md # Kotlin/Gradle build errors
|   |-- rust-reviewer.md # Rust code review
|   |-- rust-build-resolver.md # Rust build error resolution
|   |-- pytorch-build-resolver.md # PyTorch/CUDA training errors
|
|-- skills/              # Workflow definitions and domain knowledge
|   |-- coding-standards/ # Language best practices
|   |-- clickhouse-io/    # ClickHouse analytics, queries, data engineering
|   |-- backend-patterns/ # API, database, caching patterns
|   |-- frontend-patterns/ # React, Next.js patterns
|   |-- frontend-slides/  # HTML slide decks and PPTX-to-web presentation workflows (NEW)
|   |-- article-writing/   # Long-form writing in a supplied voice without generic AI tone (NEW)
|   |-- content-engine/    # Multi-platform social content and repurposing workflows (NEW)
|   |-- market-research/  # Source-attributed market, competitor, and investor research (NEW)
|   |-- investor-materials/ # Pitch decks, one-pagers, memos, and financial models (NEW)
|   |-- investor-outreach/ # Personalized fundraising outreach and follow-up (NEW)
|   |-- continuous-learning/ # Auto-extract patterns from sessions (Longform Guide)
|   |-- continuous-learning-v2/ # Instinct-based learning with confidence scoring
|   |-- iterative-retrieval/ # Progressive context refinement for subagents
|   |-- strategic-compact/ # Manual compaction suggestions (Longform Guide)
|   |-- tdd-workflow/     # TDD methodology
|   |-- security-review/  # Security checklist
|   |-- eval-harness/     # Verification loop evaluation (Longform Guide)
|   |-- verification-loop/ # Continuous verification (Longform Guide)
|   |-- videodb/          # Video and audio: ingest, search, edit, generate, stream (NEW)
|   |-- golang-patterns/  # Go idioms and best practices
|   |-- golang-testing/   # Go testing patterns, TDD, benchmarks
|   |-- cpp-coding-standards/ # C++ coding standards from C++ Core Guidelines (NEW)
|   |-- cpp-testing/      # C++ testing with GoogleTest, CMake/CTest (NEW)
|   |-- django-patterns/  # Django patterns, models, views (NEW)
|   |-- django-security/  # Django security best practices (NEW)
|   |-- django-tdd/       # Django TDD workflow (NEW)
|   |-- django-verification/ # Django verification loops (NEW)
|   |-- laravel-patterns/ # Laravel architecture patterns (NEW)
|   |-- laravel-security/ # Laravel security best practices (NEW)
|   |-- laravel-tdd/      # Laravel TDD workflow (NEW)
|   |-- laravel-verification/ # Laravel verification loops (NEW)
|   |-- python-patterns/  # Python idioms and best practices (NEW)
|   |-- python-testing/   # Python testing with pytest (NEW)
|   |-- springboot-patterns/ # Java Spring Boot patterns (NEW)
|   |-- springboot-security/ # Spring Boot security (NEW)
|   |-- springboot-tdd/   # Spring Boot TDD (NEW)
|   |-- springboot-verification/ # Spring Boot verification (NEW)
```

```

| | -- configure-ecc/          # Interactive installation wizard (NEW)
| | -- security-scan/         # AgentShield security auditor integration (NEW)
| | -- java-coding-standards/ # Java coding standards (NEW)
| | -- jpa-patterns/         # JPA/Hibernate patterns (NEW)
| | -- postgres-patterns/    # PostgreSQL optimization patterns (NEW)
| | -- nutrient-document-processing/ # Document processing with Nutrient API (NEW)
| | -- project-guidelines-example/ # Template for project-specific skills
| | -- database-migrations/   # Migration patterns (Prisma, Drizzle, Django, Go) (NEW)
| | -- api-design/           # REST API design, pagination, error responses (NEW)
| | -- deployment-patterns/   # CI/CD, Docker, health checks, rollbacks (NEW)
| | -- docker-patterns/      # Docker Compose, networking, volumes, container security (NEW)
| | -- e2e-testing/          # Playwright E2E patterns and Page Object Model (NEW)
| | -- content-hash-cache-pattern/ # SHA-256 content hash caching for file processing (NEW)
| | -- cost-aware-llm-pipeline/ # LLM cost optimization, model routing, budget tracking (NEW)
| | -- regex-vs-llm-structured-text/ # Decision framework: regex vs LLM for text parsing (NEW)
| | -- swift-actor-persistence/ # Thread-safe Swift data persistence with actors (NEW)
| | -- swift-protocol-di-testing/ # Protocol-based DI for testable Swift code (NEW)
| | -- search-first/         # Research-before-coding workflow (NEW)
| | -- skill-stocktake/      # Audit skills and commands for quality (NEW)
| | -- liquid-glass-design/   # iOS 26 Liquid Glass design system (NEW)
| | -- foundation-models-on-device/ # Apple on-device LLM with FoundationModels (NEW)
| | -- swift-concurrency-6-2/ # Swift 6.2 Approachable Concurrency (NEW)
| | -- perl-patterns/        # Modern Perl 5.36+ idioms and best practices (NEW)
| | -- perl-security/        # Perl security patterns, taint mode, safe I/O (NEW)
| | -- perl-testing/         # Perl TDD with Test2::V0, prove, Devel::Cover (NEW)
| | -- autonomous-loops/     # Autonomous loop patterns: sequential pipelines, PR loops, DAG orchestration
(NEW)
| | -- plankton-code-quality/ # Write-time code quality enforcement with Plankton hooks (NEW)
|
| -- commands/              # Slash commands for quick execution
| | -- tdd.md               # /tdd - Test-driven development
| | -- plan.md              # /plan - Implementation planning
| | -- e2e.md               # /e2e - E2E test generation
| | -- code-review.md       # /code-review - Quality review
| | -- build-fix.md         # /build-fix - Fix build errors
| | -- refactor-clean.md    # /refactor-clean - Dead code removal
| | -- learn.md             # /learn - Extract patterns mid-session (Longform Guide)
| | -- learn-eval.md        # /learn-eval - Extract, evaluate, and save patterns (NEW)
| | -- checkpoint.md        # /checkpoint - Save verification state (Longform Guide)
| | -- verify.md            # /verify - Run verification loop (Longform Guide)
| | -- setup-pm.md          # /setup-pm - Configure package manager
| | -- go-review.md         # /go-review - Go code review (NEW)
| | -- go-test.md           # /go-test - Go TDD workflow (NEW)
| | -- go-build.md          # /go-build - Fix Go build errors (NEW)
| | -- skill-create.md      # /skill-create - Generate skills from git history (NEW)
| | -- instinct-status.md   # /instinct-status - View learned instincts (NEW)
| | -- instinct-import.md   # /instinct-import - Import instincts (NEW)
| | -- instinct-export.md   # /instinct-export - Export instincts (NEW)
| | -- evolve.md            # /evolve - Cluster instincts into skills
| | -- pm2.md               # /pm2 - PM2 service lifecycle management (NEW)
| | -- multi-plan.md        # /multi-plan - Multi-agent task decomposition (NEW)
| | -- multi-execute.md     # /multi-execute - Orchestrated multi-agent workflows (NEW)
| | -- multi-backend.md     # /multi-backend - Backend multi-service orchestration (NEW)
| | -- multi-frontend.md    # /multi-frontend - Frontend multi-service orchestration (NEW)
| | -- multi-workflow.md    # /multi-workflow - General multi-service workflows (NEW)
| | -- orchestrate.md       # /orchestrate - Multi-agent coordination
| | -- sessions.md          # /sessions - Session history management
| | -- eval.md              # /eval - Evaluate against criteria
| | -- test-coverage.md     # /test-coverage - Test coverage analysis
| | -- update-docs.md       # /update-docs - Update documentation
| | -- update-codemaps.md   # /update-codemaps - Update codemaps
| | -- python-review.md     # /python-review - Python code review (NEW)
|
| -- rules/                 # Always-follow guidelines (copy to ~/.claude/rules/)
| | -- README.md            # Structure overview and installation guide
| | -- common/              # Language-agnostic principles
| | | -- coding-style.md    # Immutability, file organization
| | | -- git-workflow.md    # Commit format, PR process
| | | -- testing.md         # TDD, 80% coverage requirement
| | | -- performance.md     # Model selection, context management
| | | -- patterns.md        # Design patterns, skeleton projects
| | | -- hooks.md           # Hook architecture, TodoWrite
| | | -- agents.md          # When to delegate to subagents

```

```
| | |-- security.md          # Mandatory security checks
| | |-- typescript/         # TypeScript/JavaScript specific
| | |-- python/             # Python specific
| | |-- golang/             # Go specific
| | |-- swift/              # Swift specific
| | |-- php/                # PHP specific (NEW)
|
|-- hooks/                  # Trigger-based automations
| | |-- README.md           # Hook documentation, recipes, and customization guide
| | |-- hooks.json          # All hooks config (PreToolUse, PostToolUse, Stop, etc.)
| | |-- memory-persistence/ # Session lifecycle hooks (Longform Guide)
| | |-- strategic-compact/  # Compaction suggestions (Longform Guide)
|
|-- scripts/                # Cross-platform Node.js scripts (NEW)
| | |-- lib/                # Shared utilities
| | | |-- utils.js          # Cross-platform file/path/system utilities
| | | |-- package-manager.js # Package manager detection and selection
| | |-- hooks/              # Hook implementations
| | | |-- session-start.js  # Load context on session start
| | | |-- session-end.js    # Save state on session end
| | | |-- pre-compact.js    # Pre-compaction state saving
| | | |-- suggest-compact.js # Strategic compaction suggestions
| | | |-- evaluate-session.js # Extract patterns from sessions
| | |-- setup-package-manager.js # Interactive PM setup
|
|-- tests/                  # Test suite (NEW)
| | |-- lib/                # Library tests
| | |-- hooks/              # Hook tests
| | |-- run-all.js         # Run all tests
|
|-- contexts/               # Dynamic system prompt injection contexts (Longform Guide)
| | |-- dev.md              # Development mode context
| | |-- review.md           # Code review mode context
| | |-- research.md         # Research/exploration mode context
|
|-- examples/               # Example configurations and sessions
| | |-- CLAUDE.md           # Example project-level config
| | |-- user-CLAUDE.md      # Example user-level config
| | |-- saas-nextjs-CLAUDE.md # Real-world SaaS (Next.js + Supabase + Stripe)
| | |-- go-microservice-CLAUDE.md # Real-world Go microservice (gRPC + PostgreSQL)
| | |-- django-api-CLAUDE.md # Real-world Django REST API (DRF + Celery)
| | |-- laravel-api-CLAUDE.md # Real-world Laravel API (PostgreSQL + Redis) (NEW)
| | |-- rust-api-CLAUDE.md   # Real-world Rust API (Axum + SQLx + PostgreSQL) (NEW)
|
|-- mcp-configs/            # MCP server configurations
| | |-- mcp-servers.json    # GitHub, Supabase, Vercel, Railway, etc.
|
|-- marketplace.json        # Self-hosted marketplace config (for /plugin marketplace add)
```

Ecosystem Tools

Skill Creator

Two ways to generate Claude Code skills from your repository:

Option A: Local Analysis (Built-in)

Use the `/skill-create` command for local analysis without external services:

```
/skill-create          # Analyze current repo
/skill-create --instincts # Also generate instincts for continuous-learning
```



This analyzes your git history locally and generates SKILL.md files.

Option B: GitHub App (Advanced)

For advanced features (10k+ commits, auto-PRs, team sharing):

```
# Comment on any issue:
/skill-creator analyze

# Or auto-triggers on push to default branch
```



Both options create:

- **SKILL.md files** - Ready-to-use skills for Claude Code
- **Instinct collections** - For continuous-learning-v2
- **Pattern extraction** - Learns from your commit history

🔗 AgentShield — Security Auditor

Built at the Claude Code Hackathon (Cerebral Valley x Anthropic, Feb 2026). 1282 tests, 98% coverage, 102 static analysis rules.

Scan your Claude Code configuration for vulnerabilities, misconfigurations, and injection risks.

```
# Quick scan (no install needed)
npx ecc-agentshield scan

# Auto-fix safe issues
npx ecc-agentshield scan --fix

# Deep analysis with three Opus 4.6 agents
npx ecc-agentshield scan --opus --stream

# Generate secure config from scratch
npx ecc-agentshield init
```



What it scans: CLAUDE.md, settings.json, MCP configs, hooks, agent definitions, and skills across 5 categories — secrets detection (14 patterns), permission auditing, hook injection analysis, MCP server risk profiling, and agent config review.

The --opus flag runs three Claude Opus 4.6 agents in a red-team/blue-team/auditor pipeline. The attacker finds exploit chains, the defender evaluates protections, and the auditor synthesizes both into a prioritized risk assessment. Adversarial reasoning, not just pattern matching.

Output formats: Terminal (color-graded A-F), JSON (CI pipelines), Markdown, HTML. Exit code 2 on critical findings for build gates.

Use `/security-scan` in Claude Code to run it, or add to CI with the [GitHub Action](#).

[GitHub](#) | [npm](#)

🔗 🧪 Plankton — Write-Time Code Quality Enforcement

Plankton (credit: @alxfazio) is a recommended companion for write-time code quality enforcement. It runs formatters and 20+ linters on every file edit via PostToolUse hooks, then spawns Claude subprocesses (routed to Haiku/Sonnet/Opus by violation complexity) to fix issues the main agent missed. Three-phase architecture: auto-format silently (40-50% of issues), collect remaining violations as structured JSON, delegate fixes to a subprocess. Includes config protection hooks that prevent agents from modifying linter configs to pass instead of fixing code. Supports Python, TypeScript, Shell, YAML, JSON, TOML, Markdown, and Dockerfile. Use alongside AgentShield for security + quality coverage. See `skills/plankton-code-quality/` for full integration guide.

🔗 🧠 Continuous Learning v2

The instinct-based learning system automatically learns your patterns:

```
/instinct-status      # Show learned instincts with confidence
/instinct-import <file> # Import instincts from others
/instinct-export      # Export your instincts for sharing
/evolve              # Cluster related instincts into skills
```



See `skills/continuous-learning-v2/` for full documentation.

Requirements

Claude Code CLI Version

Minimum version: v2.1.0 or later

This plugin requires Claude Code CLI v2.1.0+ due to changes in how the plugin system handles hooks.

Check your version:

```
claude --version
```



Important: Hooks Auto-Loading Behavior

 **For Contributors:** Do NOT add a "hooks" field to `.claude-plugin/plugin.json`. This is enforced by a regression test.

Claude Code v2.1+ **automatically loads** `hooks/hooks.json` from any installed plugin by convention. Explicitly declaring it in `plugin.json` causes a duplicate detection error:

```
Duplicate hooks file detected: ./hooks/hooks.json resolves to already-loaded file
```



History: This has caused repeated fix/revert cycles in this repo ([#29](#), [#52](#), [#103](#)). The behavior changed between Claude Code versions, leading to confusion. We now have a regression test to prevent this from being reintroduced.

Installation

Option 1: Install as Plugin (Recommended)

The easiest way to use this repo - install as a Claude Code plugin:

```
# Add this repo as a marketplace
/plugin marketplace add affaan-m/everything-claude-code

# Install the plugin
/plugin install everything-claude-code@everything-claude-code
```



Or add directly to your `~/.claude/settings.json`:

```
{
  "extraKnownMarketplaces": {
    "everything-claude-code": {
      "source": {
        "source": "github",
        "repo": "affaan-m/everything-claude-code"
      }
    }
  },
  "enabledPlugins": {
    "everything-claude-code@everything-claude-code": true
  }
}
```



This gives you instant access to all commands, agents, skills, and hooks.

Note: The Claude Code plugin system does not support distributing rules via plugins ([upstream limitation](#)). You need to install rules manually:

```
# Clone the repo first
git clone https://github.com/affaan-m/everything-claude-code.git

# Option A: User-level rules (applies to all projects)
```



```

mkdir -p ~/.claude/rules
cp -r everything-claude-code/rules/common/* ~/.claude/rules/
cp -r everything-claude-code/rules/typescript/* ~/.claude/rules/ # pick your stack
cp -r everything-claude-code/rules/python/* ~/.claude/rules/
cp -r everything-claude-code/rules/golang/* ~/.claude/rules/
cp -r everything-claude-code/rules/php/* ~/.claude/rules/

# Option B: Project-level rules (applies to current project only)
mkdir -p .claude/rules
cp -r everything-claude-code/rules/common/* .claude/rules/
cp -r everything-claude-code/rules/typescript/* .claude/rules/ # pick your stack

```

🔧 Option 2: Manual Installation

If you prefer manual control over what's installed:

```

# Clone the repo
git clone https://github.com/affaan-m/everything-claude-code.git

# Copy agents to your Claude config
cp everything-claude-code/agents/*.md ~/.claude/agents/

# Copy rules (common + language-specific)
cp -r everything-claude-code/rules/common/* ~/.claude/rules/
cp -r everything-claude-code/rules/typescript/* ~/.claude/rules/ # pick your stack
cp -r everything-claude-code/rules/python/* ~/.claude/rules/
cp -r everything-claude-code/rules/golang/* ~/.claude/rules/
cp -r everything-claude-code/rules/php/* ~/.claude/rules/

# Copy commands
cp everything-claude-code/commands/*.md ~/.claude/commands/

# Copy skills (core vs niche)
# Recommended (new users): core/general skills only
cp -r everything-claude-code/.agents/skills/* ~/.claude/skills/
cp -r everything-claude-code/skills/search-first ~/.claude/skills/

# Optional: add niche/framework-specific skills only when needed
# for s in django-patterns django-tdd laravel-patterns springboot-patterns; do
#   cp -r everything-claude-code/skills/$s ~/.claude/skills/
# done

```



🔗 Add hooks to settings.json

Copy the hooks from `hooks/hooks.json` to your `~/.claude/settings.json`.

🔗 Configure MCPs

Copy desired MCP servers from `mcp-configs/mcp-servers.json` to your `~/.claude.json`.

Important: Replace `YOUR_*_HERE` placeholders with your actual API keys.

🎯 Key Concepts

🔗 Agents

Subagents handle delegated tasks with limited scope. Example:

```

---
name: code-reviewer
description: Reviews code for quality, security, and maintainability
tools: ["Read", "Grep", "Glob", "Bash"]
model: opus
---

```



You are a senior code reviewer...

🔗 Skills

Skills are workflow definitions invoked by commands or agents:

TDD Workflow

- 1. Define interfaces first
- 2. Write failing tests (RED)
- 3. Implement minimal code (GREEN)
- 4. Refactor (IMPROVE)
- 5. Verify 80%+ coverage

🔗 Hooks

Hooks fire on tool events. Example - warn about console.log:

```
{
  "matcher": "tool == \"Edit\" && tool_input.file_path matches \"\\\\\\\\\\\\\\\\.(ts|tsx|js|jsx)$\"",
  "hooks": [{
    "type": "command",
    "command": "#!/bin/bash\ngrep -n 'console\\.log' \"${file_path}\" && echo '[Hook] Remove console.log' >&2"
  }]
}
```

🔗 Rules

Rules are always-follow guidelines, organized into `common/` (language-agnostic) + language-specific directories:

```
rules/
  common/      # Universal principles (always install)
  typescript/  # TS/JS specific patterns and tools
  python/      # Python specific patterns and tools
  go/          # Go specific patterns and tools
  swift/       # Swift specific patterns and tools
  php/         # PHP specific patterns and tools
```

See [rules/README.md](#) for installation and structure details.

🔗 🌐 Which Agent Should I Use?

Not sure where to start? Use this quick reference:

I want to...	Use this command	Agent used
Plan a new feature	<code>/everything-claude-code:plan "Add auth"</code>	planner
Design system architecture	<code>/everything-claude-code:plan + architect agent</code>	architect
Write code with tests first	<code>/tdd</code>	tdd-guide
Review code I just wrote	<code>/code-review</code>	code-reviewer
Fix a failing build	<code>/build-fix</code>	build-error-resolver
Run end-to-end tests	<code>/e2e</code>	e2e-runner
Find security vulnerabilities	<code>/security-scan</code>	security-reviewer
Remove dead code	<code>/refactor-clean</code>	refactor-cleaner
Update documentation	<code>/update-docs</code>	doc-updater

I want to...	Use this command	Agent used
Review Go code	<code>/go-review</code>	go-reviewer
Review Python code	<code>/python-review</code>	python-reviewer
Review TypeScript/JavaScript code	<i>(invoke <code>typescript-reviewer</code> directly)</i>	typescript-reviewer
Audit database queries	<i>(auto-delegated)</i>	database-reviewer

Common Workflows

Starting a new feature:

```
/everything-claude-code:plan "Add user authentication with OAuth"
                                → planner creates implementation blueprint
/tdd                            → tdd-guide enforces write-tests-first
/code-review                    → code-reviewer checks your work
```



Fixing a bug:

```
/tdd                            → tdd-guide: write a failing test that reproduces it
                                → implement the fix, verify test passes
/code-review                    → code-reviewer: catch regressions
```



Preparing for production:

```
/security-scan                  → security-reviewer: OWASP Top 10 audit
/e2e                           → e2e-runner: critical user flow tests
/test-coverage                  → verify 80%+ coverage
```



? FAQ

- ▶ How do I check which agents/commands are installed?
- ▶ My hooks aren't working / I see "Duplicate hooks file" errors
- ▶ Can I use ECC with Claude Code on a custom API endpoint or model gateway?
- ▶ My context window is shrinking / Claude is running out of context
- ▶ Can I use only some components (e.g., just agents)?
- ▶ Does this work with Cursor / OpenCode / Codex / Antigravity?
- ▶ How do I contribute a new skill or agent?

Running Tests

The plugin includes a comprehensive test suite:

```
# Run all tests
node tests/run-all.js

# Run individual test files
node tests/lib/utils.test.js
node tests/lib/package-manager.test.js
node tests/hooks/hooks.test.js
```



🔗 🍌 Contributing

Contributions are welcome and encouraged.

This repo is meant to be a community resource. If you have:

- Useful agents or skills
- Clever hooks
- Better MCP configurations
- Improved rules

Please contribute! See [CONTRIBUTING.md](#) for guidelines.

🔗 Ideas for Contributions

- Language-specific skills (Rust, C#, Kotlin, Java) — Go, Python, Perl, Swift, and TypeScript already included
- Framework-specific configs (Rails, FastAPI, NestJS) — Django, Spring Boot, Laravel already included
- DevOps agents (Kubernetes, Terraform, AWS, Docker)
- Testing strategies (different frameworks, visual regression)
- Domain-specific knowledge (ML, data engineering, mobile)

🔗 Cursor IDE Support

ECC provides **full Cursor IDE support** with hooks, rules, agents, skills, commands, and MCP configs adapted for Cursor's native format.

🔗 Quick Start (Cursor)

```
# macOS/Linux
./install.sh --target cursor typescript
./install.sh --target cursor python golang swift php
```



```
# Windows PowerShell
.\install.ps1 --target cursor typescript
.\install.ps1 --target cursor python golang swift php
```



🔗 What's Included

Component	Count	Details
Hook Events	15	sessionStart, beforeShellExecution, afterFileEdit, beforeMCPExecution, beforeSubmitPrompt, and 10 more
Hook Scripts	16	Thin Node.js scripts delegating to <code>scripts/hooks/</code> via shared adapter
Rules	34	9 common (alwaysApply) + 25 language-specific (TypeScript, Python, Go, Swift, PHP)
Agents	Shared	Via AGENTS.md at root (read by Cursor natively)
Skills	Shared + Bundled	Via AGENTS.md at root and <code>.cursor/skills/</code> for translated additions
Commands	Shared	<code>.cursor/commands/</code> if installed
MCP Config	Shared	<code>.cursor/mcp.json</code> if installed

🔗 Hook Architecture (DRY Adapter Pattern)

Cursor has **more hook events than Claude Code** (20 vs 8). The `.cursor/hooks/adapter.js` module transforms Cursor's stdin JSON to Claude Code's format, allowing existing `scripts/hooks/*.js` to be reused without duplication.

Cursor stdin JSON → adapter.js → transforms → scripts/hooks/*.js
(shared with Claude Code)



Key hooks:

- **beforeShellExecution** — Blocks dev servers outside tmux (exit 2), git push review
- **afterFileEdit** — Auto-format + TypeScript check + console.log warning
- **beforeSubmitPrompt** — Detects secrets (sk-, ghp_, AKIA patterns) in prompts
- **beforeTabFileRead** — Blocks Tab from reading .env, .key, .pem files (exit 2)
- **beforeMCPExecution / afterMCPExecution** — MCP audit logging

🔗 Rules Format

Cursor rules use YAML frontmatter with `description`, `globs`, and `alwaysApply`:

```
---
description: "TypeScript coding style extending common rules"
globs: ["**/*.ts", "**/*.tsx", "**/*.js", "**/*.jsx"]
alwaysApply: false
---
```



🔗 Codex macOS App + CLI Support

ECC provides **first-class Codex support** for both the macOS app and CLI, with a reference configuration, Codex-specific AGENTS.md supplement, and shared skills.

🔗 Quick Start (Codex App + CLI)

```
# Run Codex CLI in the repo — AGENTS.md and .codex/ are auto-detected
codex

# Optional: copy the global-safe defaults to your home directory
cp .codex/config.toml ~/.codex/config.toml
```



Codex macOS app:

- Open this repository as your workspace.
- The root `AGENTS.md` is auto-detected.
- `.codex/config.toml` and `.codex/agents/*.toml` work best when kept project-local.
- The reference `.codex/config.toml` intentionally does not pin `model` or `model_provider`, so Codex uses its own current default unless you override it.
- Optional: copy `.codex/config.toml` to `~/.codex/config.toml` for global defaults; keep the multi-agent role files project-local unless you also copy `.codex/agents/`.

🔗 What's Included

Component	Count	Details
Config	1	<code>.codex/config.toml</code> — top-level approvals/sandbox/web_search, MCP servers, notifications, profiles
AGENTS.md	2	Root (universal) + <code>.codex/AGENTS.md</code> (Codex-specific supplement)
Skills	16	<code>.agents/skills/</code> — SKILL.md + agents/openai.yaml per skill
MCP Servers	4	GitHub, Context7, Memory, Sequential Thinking (command-based)
Profiles	2	<code>strict</code> (read-only sandbox) and <code>yolo</code> (full auto-approve)
Agent Roles	3	<code>.codex/agents/</code> — explorer, reviewer, docs-researcher

 Skills

Skills at `.agents/skills/` are auto-loaded by Codex:

Skill	Description
tdd-workflow	Test-driven development with 80%+ coverage
security-review	Comprehensive security checklist
coding-standards	Universal coding standards
frontend-patterns	React/Next.js patterns
frontend-slides	HTML presentations, PPTX conversion, visual style exploration
article-writing	Long-form writing from notes and voice references
content-engine	Platform-native social content and repurposing
market-research	Source-attributed market and competitor research
investor-materials	Decks, memos, models, and one-pagers
investor-outreach	Personalized outreach, follow-ups, and intro blurbs
backend-patterns	API design, database, caching
e2e-testing	Playwright E2E tests
eval-harness	Eval-driven development
strategic-compact	Context management
api-design	REST API design patterns
verification-loop	Build, test, lint, typecheck, security

 Key Limitation

Codex does **not yet provide Claude-style hook execution parity**. ECC enforcement there is instruction-based via `AGENTS.md` , optional `model_instructions_file` overrides, and sandbox/approval settings.

 Multi-Agent Support

Current Codex builds support experimental multi-agent workflows.

- Enable `features.multi_agent = true` in `.codex/config.toml`
- Define roles under `[agents.<name>]`
- Point each role at a file under `.codex/agents/`
- Use `/agent` in the CLI to inspect or steer child agents

ECC ships three sample role configs:

Role	Purpose
explorer	Read-only codebase evidence gathering before edits
reviewer	Correctness, security, and missing-test review
docs_researcher	Documentation and API verification before release/docs changes

 OpenCode Support

ECC provides **full OpenCode support** including plugins and hooks.

 Quick Start

```
# Install OpenCode
npm install -g opencode

# Run in the repository root
opencode
```



The configuration is automatically detected from `.opencode/opencode.json`.

[🔗](#) **Feature Parity**

Feature	Claude Code	OpenCode	Status
Agents	✔ 28 agents	✔ 12 agents	Claude Code leads
Commands	✔ 59 commands	✔ 31 commands	Claude Code leads
Skills	✔ 116 skills	✔ 37 skills	Claude Code leads
Hooks	✔ 8 event types	✔ 11 events	OpenCode has more!
Rules	✔ 29 rules	✔ 13 instructions	Claude Code leads
MCP Servers	✔ 14 servers	✔ Full	Full parity
Custom Tools	✔ Via hooks	✔ 6 native tools	OpenCode is better

[🔗](#) **Hook Support via Plugins**

OpenCode's plugin system is MORE sophisticated than Claude Code with 20+ event types:

Claude Code Hook	OpenCode Plugin Event
PreToolUse	tool.execute.before
PostToolUse	tool.execute.after
Stop	session.idle
SessionStart	session.created
SessionEnd	session.deleted

Additional OpenCode events: `file.edited`, `file.watcher.updated`, `message.updated`, `lsp.client.diagnostics`, `tui.toast.show`, and more.

[🔗](#) **Available Commands (31+)**

Command	Description
<code>/plan</code>	Create implementation plan
<code>/tdd</code>	Enforce TDD workflow
<code>/code-review</code>	Review code changes
<code>/build-fix</code>	Fix build errors
<code>/e2e</code>	Generate E2E tests
<code>/refactor-clean</code>	Remove dead code
<code>/orchestrate</code>	Multi-agent workflow
<code>/learn</code>	Extract patterns from session
<code>/checkpoint</code>	Save verification state
<code>/verify</code>	Run verification loop
<code>/eval</code>	Evaluate against criteria

Command	Description
/update-docs	Update documentation
/update-codemaps	Update codemaps
/test-coverage	Analyze coverage
/go-review	Go code review
/go-test	Go TDD workflow
/go-build	Fix Go build errors
/python-review	Python code review (PEP 8, type hints, security)
/multi-plan	Multi-model collaborative planning
/multi-execute	Multi-model collaborative execution
/multi-backend	Backend-focused multi-model workflow
/multi-frontend	Frontend-focused multi-model workflow
/multi-workflow	Full multi-model development workflow
/pm2	Auto-generate PM2 service commands
/sessions	Manage session history
/skill-create	Generate skills from git
/instinct-status	View learned instincts
/instinct-import	Import instincts
/instinct-export	Export instincts
/evolve	Cluster instincts into skills
/promote	Promote project instincts to global scope
/projects	List known projects and instinct stats
/learn-eval	Extract and evaluate patterns before saving
/setup-pm	Configure package manager
/harness-audit	Audit harness reliability, eval readiness, and risk posture
/loop-start	Start controlled agentic loop execution pattern
/loop-status	Inspect active loop status and checkpoints
/quality-gate	Run quality gate checks for paths or entire repo
/model-route	Route tasks to models by complexity and budget

 Plugin Installation

Option 1: Use directly

```
cd everything-claude-code
opencode
```



Option 2: Install as npm package

```
npm install ecc-universal
```



Then add to your `opencode.json` :

```
{
  "plugin": ["ecc-universal"]
}
```



That npm plugin entry enables ECC's published OpenCode plugin module (hooks/events and plugin tools). It does **not** automatically add ECC's full command/agent/instruction catalog to your project config.

For the full ECC OpenCode setup, either:

- run OpenCode inside this repository, or
- copy the bundled `.opencode/` config assets into your project and wire the `instructions` , `agent` , and `command` entries in `opencode.json`

 Documentation

- **Migration Guide:** `.opencode/MIGRATION.md`
- **OpenCode Plugin README:** `.opencode/README.md`
- **Consolidated Rules:** `.opencode/instructions/INSTRUCTIONS.md`
- **LLM Documentation:** `llms.txt` (complete OpenCode docs for LLMs)

 Cross-Tool Feature Parity

ECC is the **first plugin to maximize every major AI coding tool**. Here's how each harness compares:

Feature	Claude Code	Cursor IDE	Codex CLI	OpenCode
Agents	21	Shared (AGENTS.md)	Shared (AGENTS.md)	12
Commands	52	Shared	Instruction-based	31
Skills	102	Shared	10 (native format)	37
Hook Events	8 types	15 types	None yet	11 types
Hook Scripts	20+ scripts	16 scripts (DRY adapter)	N/A	Plugin hooks
Rules	34 (common + lang)	34 (YAML frontmatter)	Instruction-based	13 instructions
Custom Tools	Via hooks	Via hooks	N/A	6 native tools
MCP Servers	14	Shared (mcp.json)	4 (command-based)	Full
Config Format	settings.json	hooks.json + rules/	config.toml	opencode.json
Context File	CLAUDE.md + AGENTS.md	AGENTS.md	AGENTS.md	AGENTS.md
Secret Detection	Hook-based	beforeSubmitPrompt hook	Sandbox-based	Hook-based
Auto-Format	PostToolUse hook	afterFileEdit hook	N/A	file.edited hook
Version	Plugin	Plugin	Reference config	1.9.0

Key architectural decisions:

- **AGENTS.md** at root is the universal cross-tool file (read by all 4 tools)
- **DRY adapter pattern** lets Cursor reuse Claude Code's hook scripts without duplication
- **Skills format** (SKILL.md with YAML frontmatter) works across Claude Code, Codex, and OpenCode
- Codex's lack of hooks is compensated by `AGENTS.md` , optional `model_instructions_file` overrides, and sandbox permissions

 Background

I've been using Claude Code since the experimental rollout. Won the Anthropic x Forum Ventures hackathon in Sep 2025 building [zenith.chat](#) with [@DRodriguezFX](#) - entirely using Claude Code.

These configs are battle-tested across multiple production applications.

🔗 Inspiration Credits

- inspired by [zarazhangrui](#)
- homunculus-inspired by [humanplane](#)

🔗 Token Optimization

Claude Code usage can be expensive if you don't manage token consumption. These settings significantly reduce costs without sacrificing quality.

🔗 Recommended Settings

Add to `~/.claude/settings.json` :

```
{
  "model": "sonnet",
  "env": {
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }
}
```



Setting	Default	Recommended	Impact
model	opus	sonnet	~60% cost reduction; handles 80%+ of coding tasks
MAX_THINKING_TOKENS	31,999	10,000	~70% reduction in hidden thinking cost per request
CLAUDE_AUTOCOMPACT_PCT_OVERRIDE	95	50	Compacts earlier — better quality in long sessions

Switch to Opus only when you need deep architectural reasoning:

```
/model opus
```



🔗 Daily Workflow Commands

Command	When to Use
<code>/model sonnet</code>	Default for most tasks
<code>/model opus</code>	Complex architecture, debugging, deep reasoning
<code>/clear</code>	Between unrelated tasks (free, instant reset)
<code>/compact</code>	At logical task breakpoints (research done, milestone complete)
<code>/cost</code>	Monitor token spending during session

🔗 Strategic Compaction

The `strategic-compact` skill (included in this plugin) suggests `/compact` at logical breakpoints instead of relying on auto-compaction at 95% context. See `skills/strategic-compact/SKILL.md` for the full decision guide.

When to compact:

- After research/exploration, before implementation
- After completing a milestone, before starting the next
- After debugging, before continuing feature work
- After a failed approach, before trying a new one

When NOT to compact:

- Mid-implementation (you'll lose variable names, file paths, partial state)

🔗 Context Window Management

Critical: Don't enable all MCPs at once. Each MCP tool description consumes tokens from your 200k window, potentially reducing it to ~70k.

- Keep under 10 MCPs enabled per project
- Keep under 80 tools active
- Use `disabledMcpServers` in project config to disable unused ones

🔗 Agent Teams Cost Warning

Agent Teams spawns multiple context windows. Each teammate consumes tokens independently. Only use for tasks where parallelism provides clear value (multi-module work, parallel reviews). For simple sequential tasks, subagents are more token-efficient.

🔗 ⚠️ Important Notes

🔗 Token Optimization

Hitting daily limits? See the [Token Optimization Guide](#) for recommended settings and workflow tips.

Quick wins:

```
// ~/.claude/settings.json
{
  "model": "sonnet",
  "env": {
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50",
    "CLAUDE_CODE_SUBAGENT_MODEL": "haiku"
  }
}
```



Use `/clear` between unrelated tasks, `/compact` at logical breakpoints, and `/cost` to monitor spending.

🔗 Customization

These configs work for my workflow. You should:

1. Start with what resonates
2. Modify for your stack
3. Remove what you don't use
4. Add your own patterns

🔗 💜 Sponsors

This project is free and open source. Sponsors help keep it maintained and growing.

[Become a Sponsor](#) | [Sponsor Tiers](#) | [Sponsorship Program](#)

🔗 🌟 Star History

 [Star History Chart](#)

Releases 10

 **ECC v1.8.0 — Harness Performance & Cross-Platform Reliability** Latest
2 weeks ago

[+ 9 releases](#)

Sponsor this project

 **affaan-m** Affaan Mustafa



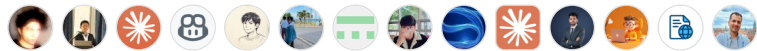
 <https://ecc.tools>

[Learn more about GitHub Sponsors](#)

Packages

No packages published

Contributors 107



[+ 93 contributors](#)

Languages

 **JavaScript** 88.6%  **Shell** 4.8%  **Python** 4.6%  **TypeScript** 2.0%